

pigzpp

Fast, Parallel, Portable Compression for the Whole Stack

Thamme Gowda

Microsoft

<https://github.com/thammegowda/pigzpp>

An experiment in AI-assisted modernization of foundational software

Affiliation for identification only; the views expressed here are the author's own.

Abstract. pigz is a widely deployed parallel gzip utility, but its process-global mutable state means that it was not designed as a reentrant, directly embeddable library. **pigzpp** is a from-scratch C++23 rewrite that turns the design into a thread-safe library with one accelerated DEFLATE core exposed to **C++, Python, WebAssembly, Go, and Rust**. As application-level conveniences built on that same core, it also provides multi-entry **ZIP** archives and a fast **PNG** codec. Its portable zlib-ng backend preserves gzip/zlib's ratio; its Intel ISA-L backend is an x86-64-only fast path that produces about 10% larger output at level 6 on our text corpus. In that configuration, zlib-ng reaches **2.1×** the CLI throughput of pigz and **16×** that of Python's in-memory gzip; ISA-L reaches **8.5×** and **50×**, respectively. Outputs remain standards-compliant and cross-decode with gzip/pigz/unzip. The result is a self-contained, multi-platform compression stack and a case study in AI-assisted modernization under automated compatibility tests.

Keywords: DEFLATE, gzip, parallel compression, zlib-ng, ISA-L, ZIP, PNG, WebAssembly, language bindings, AI-assisted modernization.

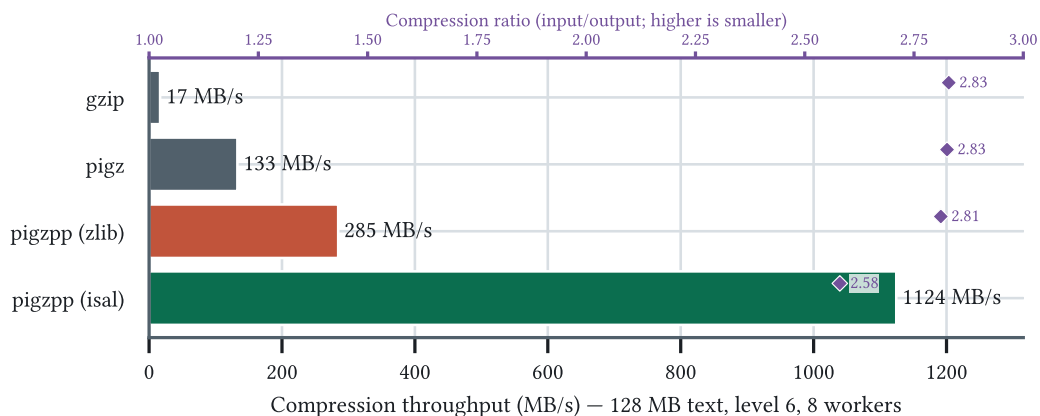


Figure 1. Headline result. CLI compression throughput on 128 MB text (level 6, 8 workers). Portable zlib-ng reaches 2.1× pigz at the same ratio; x86-64 ISA-L reaches 8.5× and produces about 9% more output.

1 Introduction

Compression is quiet infrastructure. It sits under package managers, container builds, log pipelines, scientific datasets, web assets, and archival storage—so pervasive that most of the software stack pays its cost without ever naming it. A faster compressor is therefore not a niche optimization; it is a broad, compounding speedup across everything built on top of it. And speed today means one thing above all: *parallelism*. Data volumes and CPU core counts have both grown for years while single-core speed has plateaued, so the central question for modern compression is how well it turns many cores into throughput. This paper is about parallel compression meeting modern multi-core hardware.

For three decades, one family of formats has carried the bulk of that load. The **DEFLATE** format (Deutsch, 1996a)—LZ77 (Ziv and Lempel, 1977) matching combined

with Huffman (Huffman, 1952) coding—together with gzip (Deutsch, 1996b) and its reference implementation zlib (Deutsch and Gailly, 1996), became the universal default: it is the .gz file, the common DEFLATE method in .zip archives, the compressed image-data stream in PNG, the HTTP Content-Encoding: gzip response, and the OCI/Docker image layer. Its ubiquity, not its raw ratio, is what makes it irreplaceable—anything that reads compressed data almost certainly speaks DEFLATE. Yet DEFLATE was designed to run on a single core, which puts the format the world depends on at odds with the hardware the world now runs.

pigz¹, written by Mark Adler, resolved that tension by making DEFLATE *parallel*: it splits the input into blocks, compresses them across a thread pool, and stitches the results back into a fully standard gzip stream. For over

¹pigz: <https://zlib.net/pigz/>

a decade `pigz` has been the go-to tool for saturating a multi-core machine with backward-compatible `gzip`.

Yet the shape of software has changed since `pigz` was written. Today a compression routine is as likely to be called from Python, a browser, or a Go service as from a shell, and `pigz` cannot serve those callers: its logic is entangled with a process-global struct of roughly sixty mutable fields, so it is a program, not a library. As its author put it, “*pigz is not a library. You would need to adapt the source code in `pigz` for use in other applications. That would take some work.*”² `pigzpp` asks a simple question—*what would `pigz` look like if it were designed today?*—and answers it with a thread-safe library that keeps the parallel speed, adds hardware-accelerated backends (SIMD `zlib-ng` and assembly `ISA-L`), and exposes one shared core to C++, Python, WebAssembly, Go, and Rust. ZIP and PNG conveniences apply that core to archive and image workflows rather than introducing new compression engines.

Contributions. `pigzpp` provides (1) reentrant, block-parallel `gzip` in C++23; (2) portable `zlib-ng` and x86-64 `ISA-L` behind one API; (3) bindings for five ecosystems plus application-level ZIP and PNG conveniences built on the same DEFLATE core.

Provenance. `pigzpp` is also an AI-assisted modernization experiment: much of the rewrite was performed by coding agents working under human direction, as documented in two companion essays.³ The goal was never to diminish the original authors’ work but to study how well modern agents can modernize real, load-bearing software.

2 Related Work

Parallel and accelerated DEFLATE has a rich lineage, and `pigzpp` deliberately reuses rather than reinvents the hard parts.

`pigz` pioneered block-parallel `gzip` and remains the reference for CLI throughput; `pigzpp` keeps its parallelization strategy but repackages it as a reentrant library. `zlib-ng`⁴ and `libdeflate`⁵ modernize the DEFLATE inner loops with SIMD; `pigzpp` embeds `zlib-ng` as its ratio-oriented backend rather than competing with it. Intel `ISA-L`⁶ provides hand-written assembly DEFLATE that is faster still; on the level-6 text corpus used here, its ratio trade-off means about 10% larger output than `zlib-ng`, and `pigzpp` exposes it as the default throughput backend. Per-language parallel wrappers such as `klauspost/pgzip` (Go) and `gzp` (Rust) each re-solve parallelization inside a single ecosystem; `pigzpp` instead shares one

accelerated core across five languages, so an improvement lands everywhere at once.

Newer formats such as **Zstandard** (Collet and Kuchera, 2021), **Brotli** (Alakuijala et al., 2019), and **LZ4**⁷ often beat DEFLATE on the ratio/speed frontier, but they are not wire-compatible with the enormous installed base of `gzip`, `.zip`, PNG, HTTP Content-Encoding: `gzip`, and OCI image layers. `pigzpp`’s goal is orthogonal: make the *format the world already uses* as fast as modern hardware allows, so it can be dropped in with no changes to existing readers.

3 Design and Architecture

3.1 One accelerated core, many front ends and applications

`pigzpp` centers on a single C++23 compression library with no global state. The CLI and every language binding expose that core directly; the ZIP writer and PNG encoder are application layers that call it internally. A performance improvement in the core therefore benefits both low-level APIs and these higher-level conveniences.

- **Thread-safe by construction** — configuration is passed explicitly; multiple compress/decompress operations run concurrently in one process. Threads use C++20 `std::jthread`; there is no `setjmp/longjmp` and no shared mutable state.
- **Selectable backend** — `auto` (`ISA-L` on supported x86-64 systems, otherwise `zlib-ng`), `zlib` (portable `zlib-ng`), or `isal`, chosen per call via the API or the `--engine` CLI flag.
- **Format-compatible** — the core emits standard `gzip`, `zlib`, and raw DEFLATE streams; the application helpers wrap those streams in standard ZIP or PNG structures. Existing `gzip`, `pigz`, `unzip`, and image tools can read the corresponding outputs.

3.2 Two DEFLATE backends

The core dispatches to one of two vendored, statically linked engines: **`zlib-ng`**, a SIMD-optimized drop-in for `zlib` that maximizes ratio, and Intel **`ISA-L`**, whose hand-written assembly DEFLATE maximizes throughput. For the highest compression ratio at level 11, `pigzpp` can also call Google **`Zopfli`**⁸. `zlib-ng` performs runtime CPU dispatch, so a single portable binary uses the best available SIMD path (SSE/AVX2/AVX-512 on x86, NEON on ARM) without per-CPU builds.

ISA-L is x86-64 only. `ISA-L`’s DEFLATE is hand-written x86-64 assembly, so the `ISA-L` backend is available only on x86-64; on ARM64—and in WebAssembly, which has no `ISA-L` build—`pigzpp` uses `zlib-ng`. We therefore report *both* backends throughout: the `isal` numbers are the x86-64 fast path, while the `zlib` (`zlib-ng`) numbers are the portable path every platform receives, at `gzip/zlib`’s ratio.

²M. Adler, Stack Overflow answer, <https://stackoverflow.com/a/38946216/1506477>.

³T. Gowda, *Making Compression Faster*, <https://gowda.ai/posts/2026/07/fast-compression/>; and *I Let Two AI Agents Race to Modernize `pigz`*, <https://gowda.ai/posts/2026/03/pigzpp-with-agents/>.

⁴`zlib-ng`: <https://github.com/zlib-ng/zlib-ng>

⁵`libdeflate`: <https://github.com/ebiggers/libdeflate>

⁶Intel `ISA-L`: <https://github.com/intel/isa-l>

⁷`LZ4`: <https://github.com/lz4/lz4>

⁸`Zopfli`: <https://github.com/google/zopfli>

3.3 Anatomy of a parallel compression call

To make the pipeline concrete, consider compressing a 128 MB file at level 6 with 8 threads. `pigzpp` splits the input into fixed-size blocks (128 KiB by default) and hands them to a thread pool. Each worker deflates its block independently, priming the compressor with the last 32 KiB of the previous block as a dictionary so that cross-block back-references—and thus the compression ratio—are preserved. In parallel, each worker computes the CRC-32 of its raw block; the per-block checksums are then folded into one file checksum with `crc32_combine`, which composes two CRCs in $O(\log n)$ without rescanning the data. A single writer thread emits the gzip header, streams the finished blocks strictly in input order, and appends the combined CRC and length trailer. The result is a byte-stream that any gzip or `pigz` can decode, produced while workers remain busy and without a global lock on the hot path.

3.4 Language bindings

The same core is surfaced to five ecosystems:

- **Python** via `nanobind`⁹, shipped as a single CPython stable-ABI (abi3) wheel that works on CPython 3.12 and newer.
- **WebAssembly** via Emscripten/Embind, in baseline, SIMD, and threaded (SharedArrayBuffer) variants.
- **Go** (cgo) and **Rust** (FFI) via a stable C ABI.
- **C++** directly as a static or shared library.

3.5 Applications built on DEFLATE: ZIP and PNG

ZIP and PNG are application conveniences, not additional compression backends. Both are layered on the same core. The ZIP API mirrors Python’s `zipfile`: it writes archive metadata, sends each DEFLATE member through the parallel compressor, and supports STORED entries and Zip64. The PNG codec filters scanlines and places their zlib-wrapped DEFLATE stream in IDAT chunks while handling the surrounding PNG structure. It accepts grayscale, grayscale+alpha, RGB, and RGBA buffers.

4 Benchmark Results

Native measurements were made in an Ubuntu 22.04 WSL2 guest whose host-reported CPU is an Intel Xeon W-2235. Although that processor has six physical cores and twelve hardware threads, this guest was exposed to only ten logical processors; Linux reports the allocation as one socket with five cores and two threads per core. Thus the five- and ten-worker points below represent the guest-visible core and SMT limits, not the host’s full topology. Unless a caption says otherwise, text results use a 128 MB multilingual corpus (English and Chinese Wikipedia), level 6, and eight workers. Every report benchmark performs one untimed warm-up followed by seven timed samples and uses the median as its primary result; PNG treats each complete 24-image pass as one sample. The harnesses do not standardize CPU affinity,

⁹nanobind: <https://github.com/wjakob/nanobind>

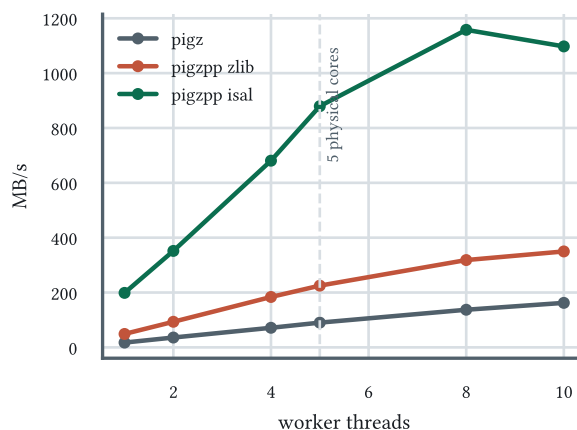


Figure 2. Native compression scaling on 128 MB text (level 6). The dashed line marks the five cores visible to the WSL2 guest; higher worker counts use its SMT threads.

turbo state, or frequency scaling. The archived result set was recorded on 2026-07-15 at commit `e7e3286b08fc`.

ratio is input/output (higher is smaller output). Because ISA-L is x86-64 only, we report both backends: `zlib` is the portable `zlib-ng` path, whereas `isal` is the additional x86-64 fast path. Across all plots, **green** denotes ISA-L, **terracotta** denotes `zlib-ng`, and gray denotes other implementations. In figures that show two metrics, bars and the lower axis encode throughput; purple diamonds, labels, and the upper axis encode compression ratio or relative output size.

4.1 Command line: pigzpp vs. gzip and pigz

Figure 1 shows that `pigzpp` `zlib-ng` outpaces `pigz` at the same ratio, while ISA-L widens the gap; bars show throughput and purple diamonds show compression ratio.

Native scaling. Figure 2 shows where the parallel implementations saturate across the five cores and ten logical CPUs visible to the WSL2 guest. In the fresh run, ISA-L peaks at eight workers (1,158 MB/s) and remains within 6% at ten, while `zlib-ng` and `pigz` continue scaling through ten workers.

4.2 Language bindings: pigzpp vs. each ecosystem

Figures 3–5 compare ecosystem-idiomatic in-memory APIs on the same 128 MB corpus (level 6, 8 workers). Bars use the lower throughput axis; purple diamonds use the upper compression-ratio axis.

Python. The portable `pigzpp` `zlib-ng` path reaches 278 MB/s, while the x86-64 ISA-L path reaches **875 MB/s**, compared with 183 MB/s for `python-isal` in this API-level comparison.

Go. The cgo paths reach **1087 MB/s** with ISA-L and 294 MB/s with portable `zlib-ng`; the subprocess path reaches 703 MB/s, including process and pipe overhead.

Rust. `zlib-ng` reaches 261 MB/s versus 225 MB/s for `gzip`; ISA-L reaches **912 MB/s**.

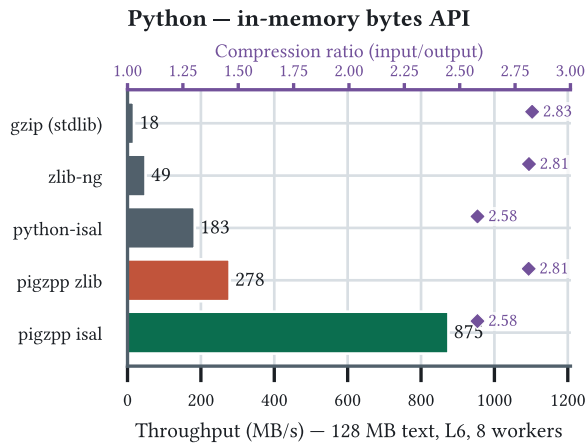


Figure 3. Python in-memory bytes API. Both pigzpp backends outperform the tested Python-native alternatives; ISA-L reaches 875 MB/s.

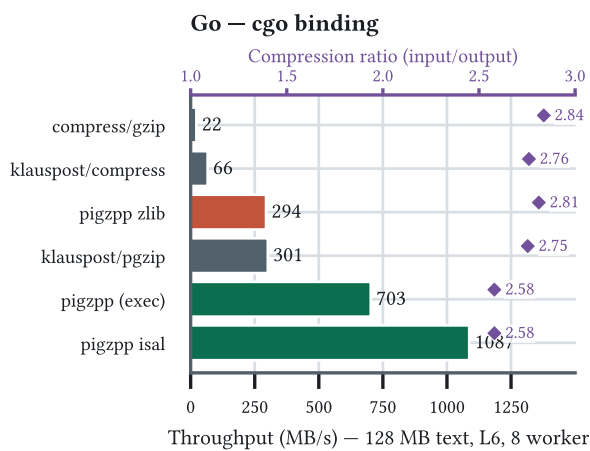


Figure 4. Go compression APIs. pigzpp’s auto/ISA-L cgo path reaches 1.1 GB/s; the subprocess path includes process and pipe overhead.

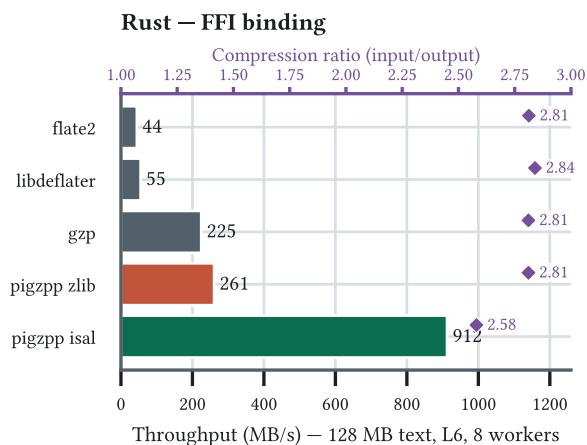


Figure 5. Rust compression APIs. pigzpp zlib-ng slightly exceeds parallel gzip; the ISA-L path reaches 912 MB/s.

4.3 Docker / OCI image layers

Image layers are gzip-compressed tarballs, and upstream BuildKit’s gzip writer uses Go’s single-threaded compress/gzip.¹⁰ Figure 6 isolates that compression stage on a real 637 MB layer (the largest layer of python:3.12); it is not an end-to-end docker build benchmark. Go

¹⁰BuildKit gzip writer: <https://github.com/moby/buildkit/blob/master/util/compression/gzip.go>

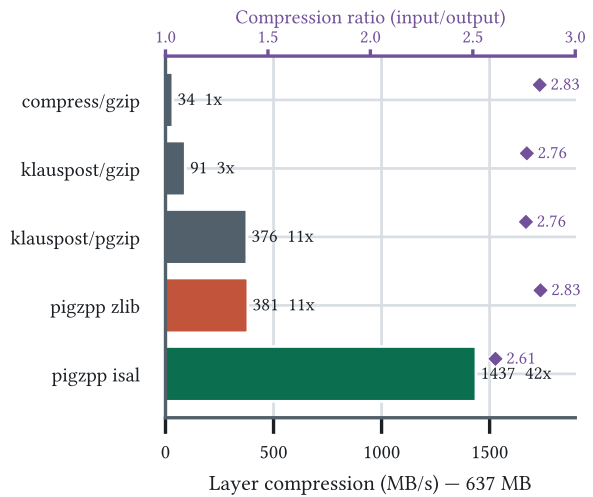


Figure 6. Compressing the largest python:3.12 image layer (637 MB); bar labels give throughput and speed-up over Go’s stdlib gzip baseline, while purple diamonds give ratio. The C-owned pigzpp zlib-ng path reaches 381 MB/s at ratio 2.83, similar to pgzip’s 376 MB/s at ratio 2.76; x86-64 ISA-L reaches 42× the stdlib throughput.

is built with its default optimizing compiler and inliner; the C++ core is a Release -O3 build. The pigzpp bars use CompressOwnedEngine, consuming the C-backed output before release instead of including an additional 225 MB C.GoBytes copy.

In this isolated stage, pigzpp compresses the layer 11× faster than the stdlib at the same ratio and 42× faster with ISA-L, reducing measured compression time from 19.6 s to 0.5 s. End-to-end build speed-up depends on the other build stages.

4.4 Level and corpus sensitivity

To test whether the level-6 text result is exceptional, we repeat the native CLI comparison at levels 1, 6, and 9 on multilingual text and incompressible random data. Figure 7 reports speed and output ratio together; in the random panel, left-facing markers at the ratio-1 boundary denote measured values slightly below 1 due to wrapper overhead.

4.5 WebAssembly

pigzpp ships as WebAssembly (zlib-ng + 128-bit SIMD; ISA-L is x86-only and thus unavailable in WASM). Figure 8 compares single-worker engines; Figure 9 measures the threaded build.

Among the implementations tested, pigzpp-wasm is fastest even with one worker, ahead of Node zlib, CompressionStream, fflate, and pako.

The threaded build reaches 3.72× speed-up with four workers (93% worker efficiency); at eight workers it uses SMT and reaches 242 MB/s, or 5.9× speed-up (74% worker efficiency).

4.6 PNG application: image encoding

PNG stores filtered scanlines in a zlib-wrapped DEFLATE stream. We compare the pigzpp.png convenience codec, which applies the shared core to this workflow, with Pillow and OpenCV on the 24-image Kodak true-color

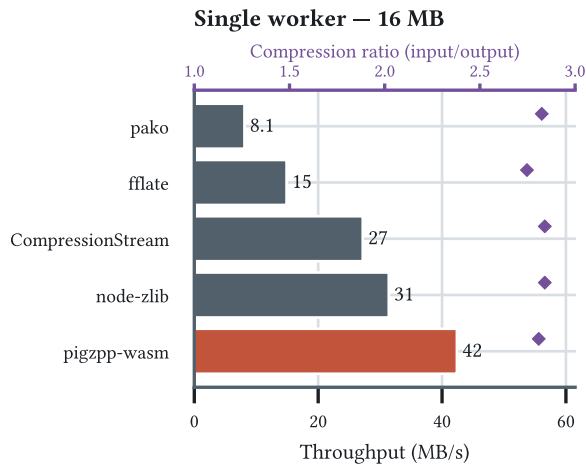


Figure 8. Single-worker WebAssembly gzip on 16 MB text (Node 22). pigzpp-wasm is fastest among the implementations tested.

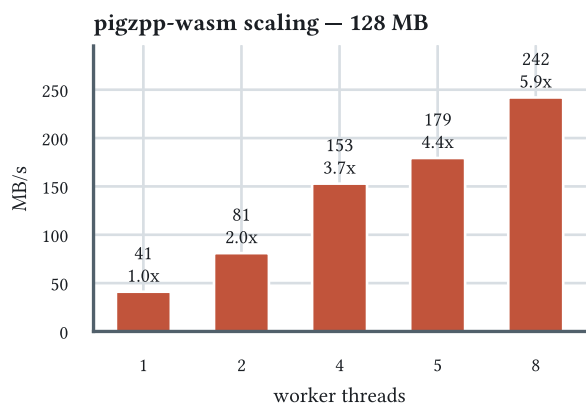


Figure 9. Threaded pigzpp-wasm on 128 MB text. Eight workers reach 242 MB/s in the WSL2 guest allocation of 5 cores/10 logical CPUs.

set¹¹ (768×512), median of seven complete passes after one warm-up, with round-trips verified. size is output relative to Pillow’s default (lower is smaller); see Figure 10. pigzpp performs the PNG line filtering itself and then DEFLATE-compresses the filtered scanlines, so it can route PNG through either backend. The default fast preset feeds the filtered scanlines to **ISA-L** (x86; zlib-ng

¹¹Kodak Lossless True Color Image Suite: <https://r0k.us/graphic/s/kodak/>

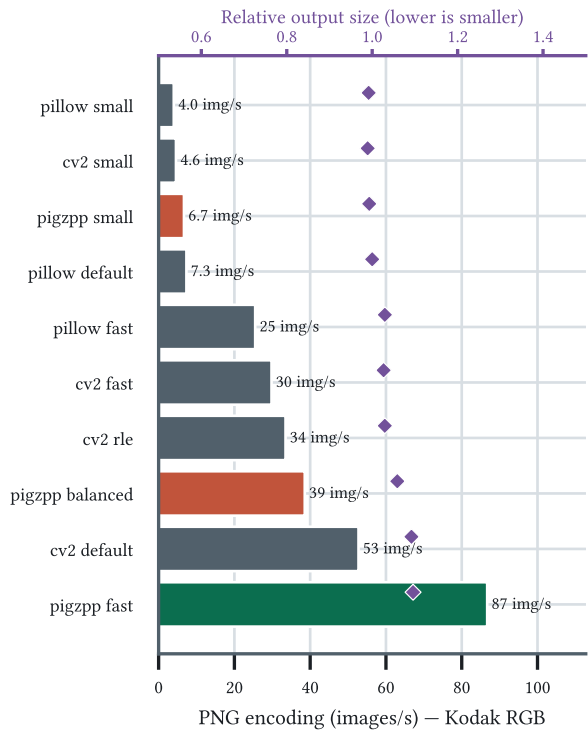


Figure 10. Application-level PNG encoding on the 24-image Kodak set (768×512), RGB. Bars give throughput; purple diamonds give output size relative to Pillow’s default. With ISA-L powering its default fast preset, pigzpp encodes at **11.8×** Pillow’s speed and **1.6×** OpenCV’s, at comparable size.

elsewhere)¹², which is $\sim 1.6\times$ faster than zlib-ng’s Z_RLE path at equal-or-better size; the higher-ratio balanced and small presets keep zlib-ng’s strategy-aware modes. pigzpp’s fast preset is the fastest PNG encoder we benchmarked on true-color images — **1.6×** faster than OpenCV (itself a hand-optimized C++ encoder) and **11.8×** faster than Pillow’s default — at a comparable size, after routing its filtered scanlines through ISA-L.

4.7 ZIP application: archives (Python)

ZIP is a container format; here pigzpp provides a convenience API that applies its parallel DEFLATE core independently to each archive member. `pigzpp.ZipFile`

¹²ISA-L is x86-64 only; on ARM64 and in WebAssembly the fast preset uses zlib-ng. A dedicated ARM64 measurement remains future work.

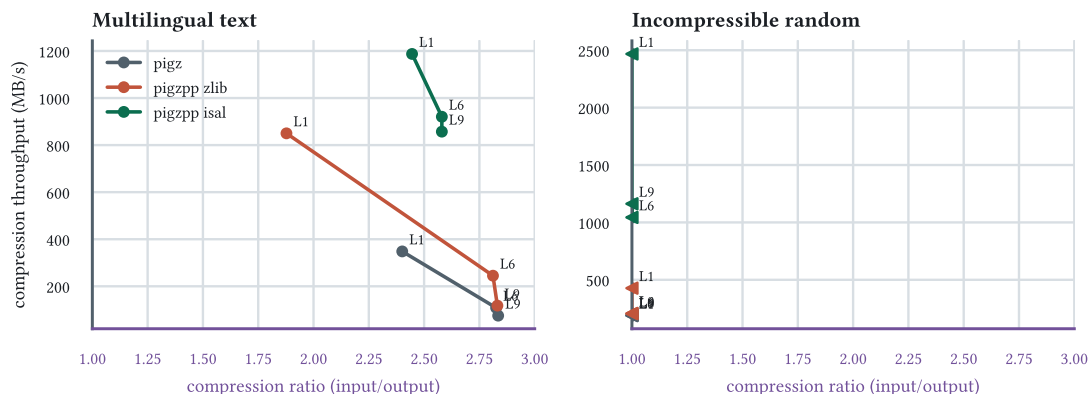


Figure 7. Compression throughput versus ratio for levels 1, 6, and 9 on 128 MB text and random corpora. Each point is the median of seven timed runs.

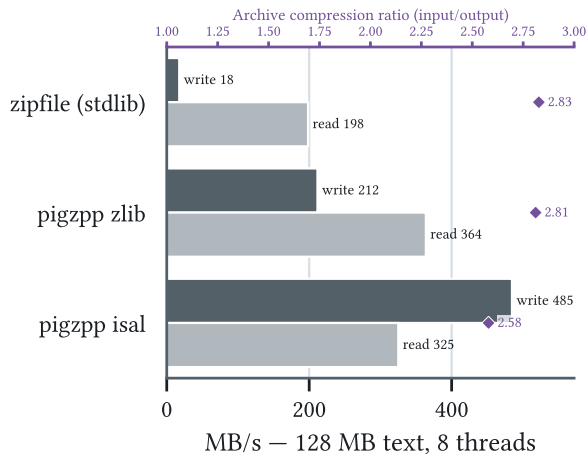


Figure 11. Application-level ZIP archives in Python (128 MB text, 8 threads). Write/read bars use the lower throughput axis; purple diamonds give archive ratio. pigzpp applies its parallel DEFLATE core to each member, writing 12–28× faster than the standard library.

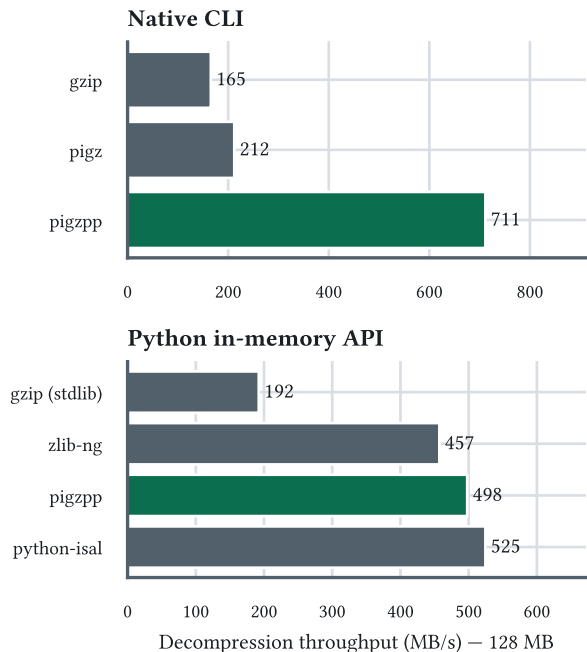


Figure 12. Decompression throughput on 128 MB text. Native tools decode the same stdlib gzip stream; the Python panel compares in-memory APIs.

vs. the standard library’s zipfile, both writing real DEFLATE archives and reading them back (128 MB text, 8 threads); results are in Figure 11.

pigzpp parallelizes each member, writing 12× faster than zipfile at the same ratio (zlib) and up to 28× faster with ISA-L, while reading up to 1.8× faster.

4.8 Decompression

The same report run also measures decompression of a common level-6 gzip stream through native CLIs and Python in-memory APIs.

Figure 12 separates decompression performance from the encoder-specific speed/ratio trade-offs shown above. Native pigzpp reaches 711 MB/s, versus 212 MB/s for pigz and 166 MB/s for gzip; the Python in-memory results range from 192 MB/s for the standard library

to 525 MB/s for python-isal, with pigzpp at 498 MB/s. Python decompression is single-threaded: its speed comes from optimized inflate, and the binding writes directly into the result bytes allocation to avoid a full-output copy.

5 Where pigzpp Helps

- **Command-line and pipelines** — a drop-in gzip/pigz replacement for build systems, backups, and log rotation.
- **Container builds** — faster OCI/Docker layer compression through cgo; overall build speed-up depends on non-compression stages.
- **Data and ML** — high-throughput gzip/zlib from Python without leaving the process, plus a PNG convenience layer that applies the same DEFLATE core to image pipelines.
- **Archives** — a zipfile convenience layer that applies the core per member and produces standard, interoperable ZIPs.
- **Browser and edge** — WebAssembly builds that outperform native CompressionStream and popular JS libraries for both DEFLATE and ZIP.
- **Services in Go and Rust** — one accelerated core behind idiomatic bindings, replacing per-language parallel-gzip libraries.

6 Packaging and Portability

pigzpp is engineered to ship as self-contained artifacts. Releases include a native CLI archive and a Python abi3 wheel for each of {Linux, macOS, Windows} × {x86-64, ARM64}, plus three WebAssembly variants. A single stable-ABI wheel serves CPython 3.12 and every later release, subject to that ABI’s platform support, removing the usual per-version wheel matrix.

Crucially, every artifact **embeds its compression engines statically**: zlib-ng, ISA-L (where its assembly is supported), Zopfli, and the binding runtime are linked in, so no artifact requires those libraries to be installed at run time. Linux CLI binaries are fully static; Windows binaries use the static MSVC runtime; Linux wheels additionally embed the GNU C++/GCC runtimes. Continuous integration inspects every final archive and repaired wheel and *rejects* any dynamic dependency on zlib-ng or ISA-L, guaranteeing the self-contained property rather than merely assuming it.

7 Reproducibility

Each reported value originates in a harness under benchmarks/: core, Python, PNG, Go/Docker, Rust, or WASM. Core, language-binding, ZIP, and WASM experiments share corpora generated by benchmarks/core/gen_data.py; Docker uses a real image layer, and PNG uses the Kodak image set. Representative commands are:

```
make bench-bin # CLI: gzip vs pigz vs pigzpp
make bench-py # Python: gzip/zlib-ng/isal/pigzpp
make bench-png # PNG: pigzpp vs Pillow / OpenCV
```

Absolute throughput varies with CPU, scheduler, frequency state, and library versions. The TSV files under `report/plots/data/` are generated from archived raw JSON under `benchmarks/results/`; the JSON records every timed sample, exact command, commit state, machine/runtime metadata, corpus sizes and SHA-256 hashes, output sizes, and ratios. `make plots` regenerates all figures, and `benchmarks/report/validate_report_data.py` rejects missing samples or TSVs that disagree with the raw results.

8 Limitations and Threats to Validity

Native results come from one x86-64 WSL2 environment with a 10-vCPU guest allocation; no native ARM64 throughput is reported, and ISA-L results do not generalize beyond x86-64. Most compression comparisons use one 128 MB multilingual-text corpus at level 6, so speed and ratio may differ for source code, binary data, incompressible input, and other levels. The language panels compare ecosystem-idiomatic APIs rather than isolating binding overhead. No common CPU affinity, turbo, or frequency-control policy is enforced. Accordingly, “fastest” means fastest among the implementations, inputs, and configurations tested here; it is not a universal ranking.

9 Lessons Learned

Two lessons stand out from rebuilding a mature, load-bearing C program with AI coding agents.

AI agents excel when feedback is tight. Progress was fastest where correctness was mechanically checkable: lossless round-trips, cross-decoding with gzip and unzip, a large test suite, and repeatable benchmarks. Agents propose changes quickly; automated oracles are what make accepting those changes safe.

Global state is the real portability barrier. The hardest part of the rewrite was not performance but untangling pigz’s process-global struct `g`. Removing it—threading configuration explicitly and giving each operation its own state—is what turned a program into a library and unlocked every binding. Performance largely took care of itself once the vendored engines were in place.

10 Conclusion

pigzpp shows that a decades-old, ubiquitous format need not be slow. By pairing pigz’s parallel design with SIMD and assembly DEFLATE backends and exposing one thread-safe core to five languages, its portable zlib-ng path reaches up to **2.1×** the CLI throughput of pigz and **16×** that of Python’s in-memory gzip; its x86-64-only ISA-L path reaches **8.5×** and **50×**, respectively. Its output remains standards-compliant and interoperable as gzip and through the ZIP and PNG application layers built on the same DEFLATE core. Release artifacts embed pigzpp’s compression engines and relevant C++ runtime dependencies, while using required platform system libraries. The benchmark harnesses and exact plot inputs are included so the evaluation can be inspected

and extended.

11 Acknowledgments and Credits

pigzpp stands entirely on the shoulders of prior work, and this section is the most important one.

- **Mark Adler** — co-creator of zlib, gzip, and the DEFLATE format (Deutsch and Gailly, 1996; Deutsch, 1996b,a), and author of pigz. pigzpp is a rewrite of his design and exists only because of the decades of work he invested in building and maintaining these foundations. This is an *altered* version; it is not the original pigz and must not be mistaken for it.
- **Marcin Junczys-Dowmunt** — for valuable discussion, and in particular for suggesting that combining pigz with Intel ISA-L could yield large speedups on Intel x86 hardware, an idea that became pigzpp’s default throughput backend.
- **zlib-ng contributors** — the SIMD-optimized, runtime-dispatched DEFLATE engine that gives pigzpp its portable acceleration.
- **Intel ISA-L** — the hand-tuned assembly DEFLATE that powers pigzpp’s fastest path.
- **Google Zopfli** — optimal DEFLATE for the highest-ratio level.

The design rationale, the AI-assisted process, and the deeper performance analysis are described in the two companion essays footnoted in the introduction. pigzpp is released under the zlib license, the same license as the original pigz.

References

- Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltán Szabadka, and Lode Vandevenne. 2019. [Brotli: A general-purpose data compressor](#). *ACM Transactions on Information Systems*, 37(1):4:1–4:30.
- Yann Collet and Murray S. Kucherawy. 2021. [Zstandard compression and the application/zstd media type](#). Request for Comments RFC 8878, Internet Engineering Task Force.
- L. Peter Deutsch. 1996a. [DEFLATE compressed data format specification version 1.3](#). Request for Comments RFC 1951, Internet Engineering Task Force.
- L. Peter Deutsch. 1996b. [GZIP file format specification version 4.3](#). Request for Comments RFC 1952, Internet Engineering Task Force.
- L. Peter Deutsch and Jean-Loup Gailly. 1996. [ZLIB compressed data format specification version 3.3](#). Request for Comments RFC 1950, Internet Engineering Task Force.
- David A. Huffman. 1952. [A method for the construction of minimum-redundancy codes](#). *Proceedings of the IRE*, 40(9):1098–1101.
- Jacob Ziv and Abraham Lempel. 1977. [A universal algorithm for sequential data compression](#). *IEEE Transactions on Information Theory*, 23(3):337–343.